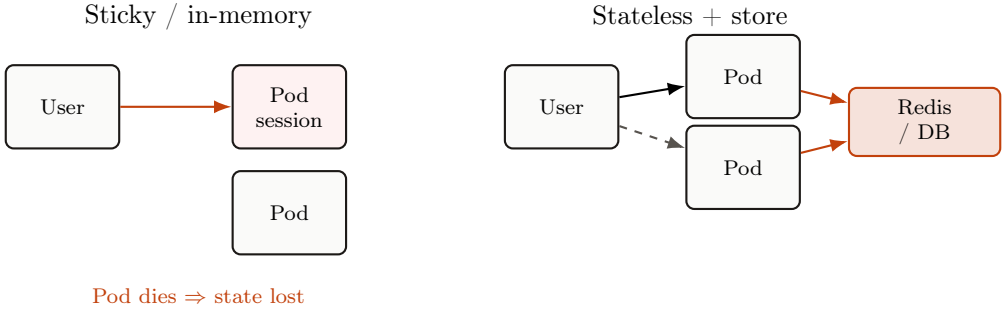
Stateless Design, Discovery, and Resilience

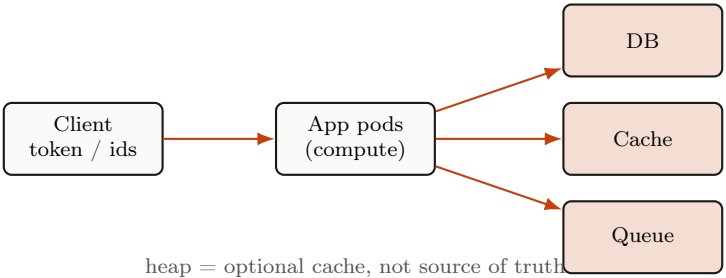
Analysis and Assessment of Design Quality

Stateful vs stateless replicas



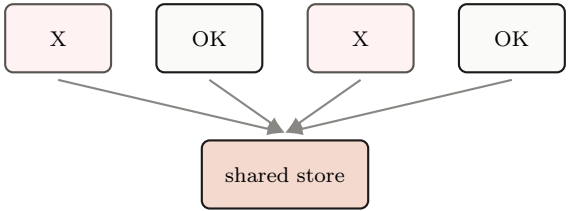
Any healthy replica can serve the next request.

Where state lives



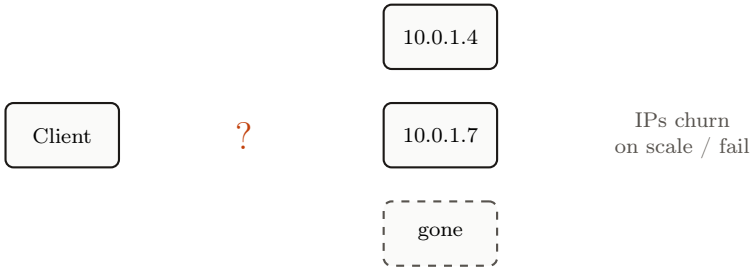
heap = optional cache, not source of truth

Kill 50% of pods — who survives?



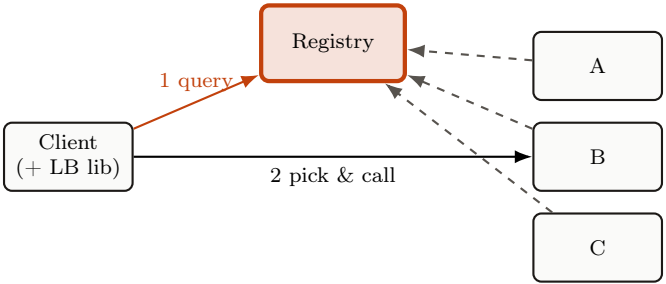
Stateless claim is only true if the store holds the session/cart truth.

The discovery problem



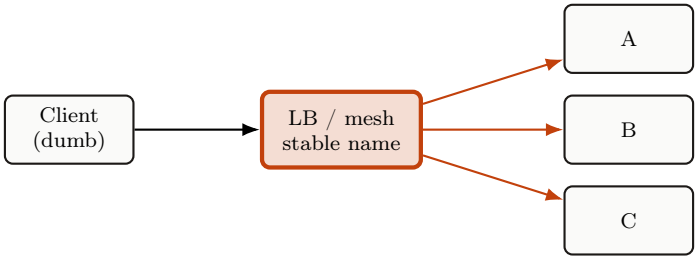
Where are healthy instances of service X — now?

Client-side discovery



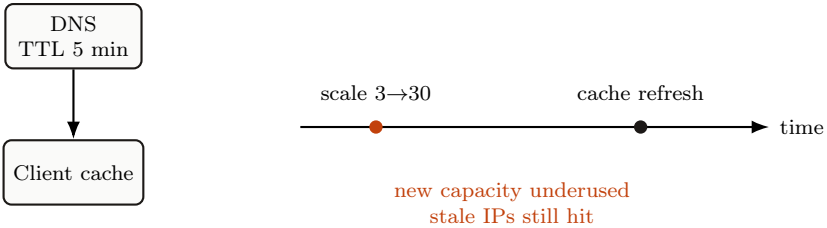
Pros: fewer hops. Cons: every stack needs the library.

Server-side discovery



Pros: central policy. Cons: LB/mesh is critical path — needs HA.

DNS discovery + TTL lag



Quiz — discovery

Quiz

Pods 3→30 in two minutes; clients cache DNS 5 minutes. Likely defect?

1. Horizontal scaling is impossible
2. Discovery freshness / TTL mismatch
3. Round-robin forbidden with DNS
4. Circuit breakers replace discovery

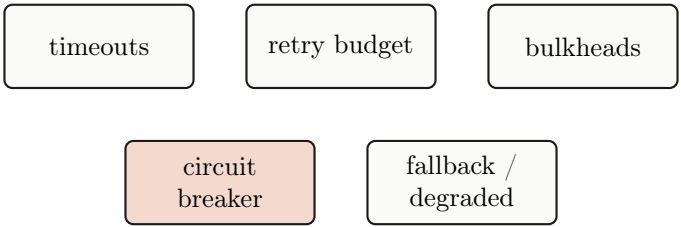
Pause — discuss with a neighbour or answer privately before the next slide.

Answer — discovery

Answer

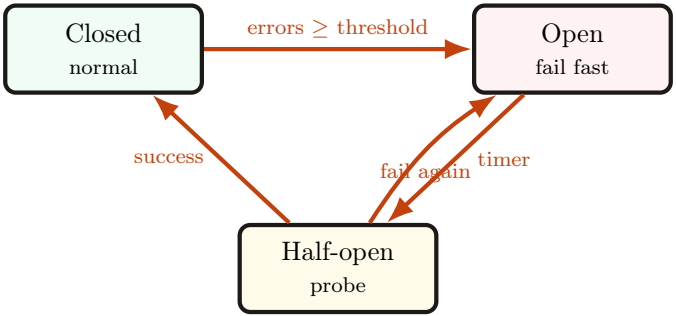
2. Discovery (or LB targets) must update on the same timescale as churn.

Resilience toolkit (map)



Remotes fail — unbounded retries make outages worse.

Circuit breaker state machine



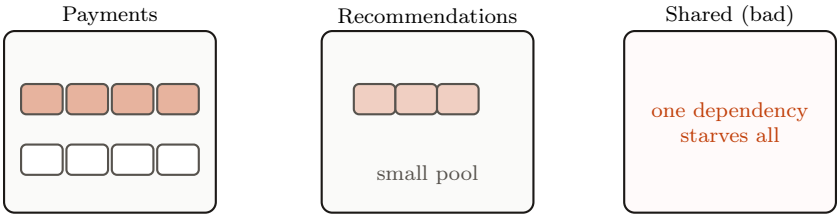
Retry storm (amplifier)



each layer retries 3×, no shared budget

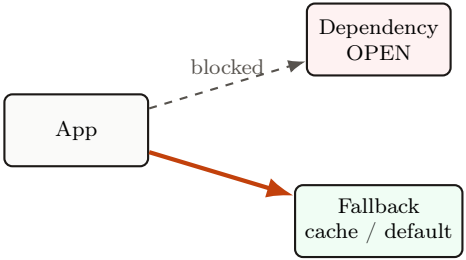
Fix: deadlines, idempotency, budget + jitter, align LB retries.

Bulkhead pools



Separate pools / queues so one sick dependency cannot consume everything.

Fallback when open



must match
quality scenario

Quiz — resilience

Quiz

Checkout→Inventory (2 s). App retries 5× immediately; LB retries too. Inventory overloaded. Finding?

1. Excellent — many retries mean success
2. Major smell: amplifying retries; need budget, backoff, breaker, aligned LB
3. Only a usability issue
4. Fixed by vertical scale alone

Pause — discuss with a neighbour or answer privately before the next slide.

Answer — resilience

Answer

2. Retries without budget are a cascade amplifier; capacity alone does not fix policy.

Review snapshot

